PÓS TECH

DEVOPS E ARQUITETURA CLOUD

FASE 4 | AULA 01 -

DATADOG - CONFIG INICIAL, INFRA HOST, CONTAINERS E AGENT

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3
HANDS ON 4
SAIBA MAIS 5
MERCADO, CASES E TENDÊNCIAS 31
O QUE VOCÊ VIU NESTA AULA? 32
REFERÊNCIAS 33

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O QUE VEM POR AÍ?

Nesta primeira aula, vamos estabelecer os fundamentos que sustentam toda a disciplina de APM. Você entenderá **a diferença profunda entre monitoramento e observabilidade**, a importância estratégica dos **três pilares (métricas, logs e traces)** e o papel crucial da **alta cardinalidade** para responder perguntas complexas de produção. Também conhecerá a **arquitetura lógica do Datadog** — desde a coleta no Agent, passando pelos pipelines de processamento, até a visualização na plataforma — e compreenderá como o Datadog se encaixa em ambientes modernos baseados em cloud, containers e microsserviços.

Vamos destrinchar a **arquitetura interna do Datadog Agent**, compreender como seus componentes (Collector, Forwarder, DogStatsD, Log Agent, Trace Agent) se articulam, e preparar o terreno para o que virá nas próximas aulas: instrumentação de aplicações, tracing distribuído e correlação de sinais.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

HANDS ON

Nesta aula, você irá instalar e validar o Datadog Agent em dois ambientes reais: **Host Linux e Docker**. Cada etapa é prática, guiada e detalhada.

Para configurar o Datadog em ambiente **Linux**, comece criando uma instância e executando o comando oficial de instalação do Agent. É fundamental configurar corretamente as variáveis DD_API_KEY e DD_SITE, além de validar o arquivo datadog.yaml para garantir a comunicação. Caso surjam problemas de conectividade ou chaves inválidas, utilize os comandos de diagnóstico status e flare para investigar falhas e validar o estado do serviço.

No cenário **Docker**, o Agent deve ser executado como container, exigindo a compreensão detalhada dos volumes montados e a ativação dos recursos de logs, métricas e autodiscovery. Para testar na prática, construa uma stack via docker-compose integrando uma aplicação de teste, Redis e o Agent. Por fim, verifique se o autodiscovery está detectando os serviços automaticamente e se as métricas estão sendo coletadas e enviadas conforme o esperado.

Ao final, você terá **um ambiente Datadog funcional**, capaz de coletar métricas de infraestrutura, containers e processos em tempo real.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

SAIBA MAIS

Monitoramento vs. Observabilidade: Uma Mudança de Paradigma

O monitoramento tradicional é, em sua essência, um sistema de verificação de hipóteses pré-definidas. Nós, como engenheiros(as), antecipamos os modos de falha ("o que pode dar errado?") e criamos um dashboard ou um alerta para cada um. Isso nos permite responder a perguntas que já sabemos que precisaremos fazer. São os "known unknowns" (sabemos que não sabemos qual será o uso da CPU, então o monitoramos).

A observabilidade, por outro lado, é sobre equipar o sistema com as ferramentas para responder a perguntas que nunca imaginamos fazer. É sobre explorar o comportamento do sistema em produção para descobrir os "unknown unknowns" (as falhas e interações complexas que não poderíamos prever). Um sistema é observável se, a partir de seus dados externos (a telemetria que ele emite), podemos inferir seu estado interno e entender qualquer comportamento, por mais bizarro que seja.

Dimensão	Monitoramento Tradicional (O Relógio)	Observabilidade Moderna (A Floresta)
Objetivo	Verificar a saúde de componentes individuais.	Entender o comportamento do sistema como um todo.
Abordagem	Reativa. Baseada em dashboards e alertas pré-definidos.	Proativa e exploratória. Baseada em depuração e análise de dados.
Dados	Métricas de baixa cardinalidade (ex: `cpu.total.user`).	Telemetria de alta cardinalidade (ex: `request.latency` com tags para `customer`, `region`, `version`).
Exemplo de Pergunta	"A CPU do servidor web está acima de 90%?"	"Quais clientes específicos estão experienciando latência acima de 2s no endpoint de checkout desde o último deploy?"

Figura 1 - Os Três Pilares da Observabilidade: Métricas, Logs e Traces
Fonte: Elaborado pelo autor (2026)

A observabilidade de um sistema é sustentada por três tipos fundamentais de dados de telemetria, conhecidos como os **Três pilares da observabilidade: Métricas, Logs e Traces.** Cada pilar oferece uma perspectiva única sobre o comportamento e a saúde do sistema. É a combinação e a correlação entre eles que permitem uma compreensão profunda e a capacidade de diagnosticar problemas

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

complexos em ambientes distribuídos. Sozinhos, eles fornecem uma visão limitada; juntos, eles formam uma narrativa completa do que está acontecendo dentro de uma aplicação.

Métricas: O "O Quê" Agregado

As **métricas** são representações numéricas de dados medidos ao longo do tempo. Elas são a base do monitoramento tradicional e fornecem uma visão quantitativa e agregada da saúde do sistema. Métricas são eficientes para armazenar e consultar, tornando-as ideais para a criação de dashboards e alertas sobre o comportamento do sistema em um nível macro. Elas nos dizem *o que* está acontecendo.

Por serem agregadas, as métricas perdem a granularidade de eventos individuais. Por exemplo, uma métrica pode nos dizer que a latência média de uma API foi de 200ms na última hora, mas não pode nos dizer por que uma solicitação específica demorou 5 segundos. Elas são perfeitas para identificar tendências, anomalias e para sinalizar que um problema existe.

Logs: O "Porquê" Detalhado

Os logs são registros de eventos discretos, imutáveis e com carimbo de data/hora. Cada log representa um evento específico que ocorreu em um ponto no tempo, como um erro, uma transação concluída ou o início de um processo. Os logs fornecem o contexto detalhado e granular que as métricas não possuem. Eles são a fonte da verdade para entender *por que* algo aconteceu.

Logs Não Estruturados (Plain Text):

INFO: 2025-11-30 10:15:02 - User 12345 successfully processed payment for order 98765.

- **Prós:** fáceis de escrever e ler para um humano.
- **Contras:** extremamente difíceis de analisar por uma máquina. Para encontrar todos os logs do usuário 12345, você precisa fazer uma busca de

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

texto completo, o que é lento e propenso a erros. E se você quiser agregar o valor dos pedidos processados? Impossível.

Logs Estruturados (JSON):

{ "timestamp": "2025-11-30T10:15:02Z", "level": "INFO", "message": "Payment processed", "user_id": "12345", "order_id": "98765", "amount": 99.90 }

- **Prós:** ricos em contexto e fáceis de analisar por uma máquina. Cada campo (como user_id e amount) pode ser indexado. Você pode fazer buscas precisas (@user_id:12345), criar agregações (sum(@amount)), e criar dashboards e alertas baseados em qualquer campo.
- **Contras:** um pouco mais verbosos e podem ser ligeiramente mais difíceis de ler para um humano no formato bruto.

A recomendação é clara: sempre que possível, use logs estruturados. O ganho em capacidade de análise e troubleshooting supera em muito a pequena sobrecarga na escrita e no armazenamento.

Traces: O "Onde" Distribuído

Os traces (ou rastreamentos distribuídos) registram a jornada de uma solicitação à medida que ela viaja através dos múltiplos serviços em uma arquitetura de microsserviços. Um trace é composto por uma série de *spans*, onde cada span representa uma unidade de trabalho ou uma operação específica (por exemplo, uma chamada de API, uma consulta ao banco de dados). Os traces são indispensáveis para entender o fluxo do sistema e identificar *onde* ocorrem gargalos de latência ou erros em um ambiente distribuído.

Um trace é o que une a história de uma única requisição enquanto ela viaja através de múltiplos serviços. É a ferramenta mais importante para entender a latência e as dependências em uma arquitetura de microsserviços.

- O **Trace** inteiro representa a ação "Comprar". Ele tem um Trace ID único.
- Cada caixa é um **Span**, representando uma operação. Cada span tem seu próprio Span ID.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- A relação entre eles é definida pelo Parent ID. O Parent ID do Span C é o Span ID do Span B.

O **Flame Graph** que veremos nas próximas aulas é uma visualização deste diagrama, onde a largura de cada span representa sua duração.

Pilar	O Que Responde	Natureza	Exemplo de Uso
Métricas	**O quê?** (Visão agregada)	Numérica, agregada	"A taxa de erro do serviço de pagamento aumentou para 5% nos últimos 10 minutos."
Logs	**Por quê?** (Contexto detalhado)	Evento, granular	"O serviço de pagamento está falhando devido a um erro de 'conexão recusada' ao tentar acessar o gateway de pagamento externo."
Traces	**Onde?** (Fluxo e latência)	Transação, distribuída	"A latência no serviço de pagamento está concentrada na chamada para a API de validação de fraude, que está demorando 3 segundos para responder."

Figura 2 - Comparativo entre Métricas, Logs e Traces
Fonte: Elaborado pelo autor (2026)

O Primeiro Pilar: Métricas e o Poder da Alta Cardinalidade

Uma métrica é uma medida numérica de algo ao longo do tempo. Mas no contexto da observabilidade, uma métrica é muito mais. É um ponto de dados com um nome, um timestamp, um valor e, o mais importante, um conjunto de tags (ou labels).

Cardinalidade é o número de valores únicos em um conjunto. Em métricas, a cardinalidade se refere ao número de combinações únicas de tags que uma métrica pode ter.

- **Baixa Cardinalidade:** uma métrica como system.cpu.total tem baixa cardinalidade. Ela pode ter, no máximo, algumas tags como host e env. O número de combinações é limitado.
- **Alta Cardinalidade:** uma métrica como http.request.latency pode ter tags como host, env, service, endpoint, http_method, customer_id, user_agent, region, availability_zone, etc. Se você tem 1000 clientes, 10 endpoints e 5 regiões, o número de combinações possíveis já é de 50.000. Isso é alta cardinalidade.

Por que a alta cardinalidade é a chave para a observabilidade? Porque ela permite que você fatie e analise os dados de formas quase infinitas, permitindo que você faça perguntas exploratórias e chegue à causa raiz de problemas complexos. Sem alta cardinalidade, você só pode ver a latência média de todas as requisições.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Com alta cardinalidade, você pode ver a latência apenas para o customer_id: 12345 no endpoint /api/v3/cart na região us-east-1.

Arquitetura Lógica do Datadog

Podemos visualizar o fluxo de dados no Datadog em três grandes estágios:

Estágio 1 (Coleta): toda a telemetria é produzida dentro da infraestrutura do cliente por meio de agentes distribuídos – Datadog Agent em hosts e containers, extensões serverless em Lambdas, além de integrações e bibliotecas como DogStatsD. Esses componentes são responsáveis por capturar métricas de sistema, logs, traces, processos em execução e telemetria customizada enviada pela aplicação. Todos os dados coletados são transmitidos de forma segura para a plataforma Datadog via TLS/HTTPS, compondo o Data Plane de observabilidade que traduz o comportamento real da infraestrutura e dos serviços.

Estágio 2 (Plataforma Datadog): ocorre o processamento inteligente dessa telemetria. A nuvem da Datadog executa a ingestão, parsing, enriquecimento e indexação de logs, métricas e traces, aplicando retenção hot/cold conforme políticas de armazenamento. Nesse estágio também opera o motor de alertas, detecção de anomalias e correlação (Watchdog/AIOps), capaz de identificar padrões inesperados e gerar insights operacionais.

Estágio 3 (Visualização e Interação), o usuário consome essas informações por meio da interface web: dashboards, explorador de logs, mapas de serviços, traces e monitores. Essa camada é um front-end sobre APIs que consultam o processamento realizado no estágio anterior, permitindo que o(a) engenheiro(a) investigue incidentes, monitore SLIs/SLOs, identifique causas-raiz e tome decisões baseadas em dados. Assim, os três estágios formam um ciclo contínuo que transforma sinais brutos da infraestrutura em inteligência operacional aplicável.

Componentes Internos do Agent

O Datadog Agent, a partir da versão 6, foi reescrito em Go para melhorar a performance e a eficiência, e possui uma arquitetura modular projetada para lidar com a coleta de diversos tipos de telemetria de forma simultânea e com baixo overhead. Embora opere como um único serviço na maioria dos sistemas, o Agent é composto

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

por vários processos e componentes internos especializados, cada um com uma responsabilidade clara.

O Agent principal (processo agent) é o orquestrador central, mas ele delega tarefas a outros processos e subcomponentes. Os principais componentes são:

Componente	Processo Principal	Responsabilidade Primária
Collector	`agent`	Executar checks de integrações (ex: Redis, NGINX) e coletar métricas.
Forwarder	`agent`	Serializar, comprimir e enviar payloads de métricas e eventos via HTTPS.
DogStatsD	`agent`	Receber métricas customizadas via UDP, agregá-las e passá-las para o Forwarder.
Trace Agent	`trace-agent`	Receber traces de APM das bibliotecas de instrumentação e enviá-los ao Datadog.
Log Agent	`agent`	Coletar logs de arquivos, sockets ou do sistema e enviá-los ao Datadog.

Figura 3 – Processo Agent
Fonte: Elaborado pelo autor (2026)

The Collector: o coração da coleta de métricas baseada em integrações. Ele executa periodicamente (por padrão, a cada 15 segundos) os *checks* configurados nos arquivos YAML de cada integração (localizados em conf.d/). Cada check é uma pequena peça de código, geralmente em Python, que sabe como se conectar a um serviço específico (como um banco de dados PostgreSQL ou um servidor web NGINX) e coletar métricas de performance relevantes.

The Forwarder: atua como o despachante de dados do Agent. Sua função é pegar os dados serializados de métricas, eventos e service checks, agrupá-los em payloads eficientes, comprimi-los e enviá-los de forma segura para a API do Datadog via HTTPS. Ele possui mecanismos de buffer e retry para lidar com interrupções de rede, garantindo que a telemetria não seja perdida durante falhas de conectividade temporárias. Os dados são mantidos em memória até um certo limite, após o qual os dados mais antigos podem ser descartados para gerenciar o consumo de memória do Agent.

DogStatsD: componente extremamente importante que roda dentro do processo principal do Agent. Ele implementa o protocolo StatsD da Etsy com extensões do Datadog. Ele escuta em uma porta UDP (padrão 8125) por métricas, eventos e service checks enviados por aplicações. Por usar UDP, o envio de métricas é um processo *fire-and-forget*, o que significa que a aplicação não bloqueia nem espera por uma resposta, garantindo que a instrumentação não adicione latência à

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

aplicação. DogStatsD agrega os dados recebidos em um intervalo de tempo (o *flush interval*, de 10 segundos) antes de passá-los ao Forwarder. Este componente será explorado em detalhe no próximo subtópico.

Trace Agent: é um processo separado (trace-agent) que é habilitado por padrão. Sua única responsabilidade é lidar com dados de APM (traces). As bibliotecas de instrumentação do Datadog (como a Datadog.Trace para .NET) enviam os traces para o Trace Agent (padrão na porta 8126/tcp). O Trace Agent então processa esses traces, realiza amostragem (sampling) para gerenciar o volume e os encaminha para o endpoint de APM do Datadog. Separar o processamento de traces em seu próprio processo isola os recursos e garante que a coleta de traces de alto volume não interfira na coleta de métricas de infraestrutura.

Log Agent: é responsável por toda a coleta de logs. Ele pode ser configurado para "seguir" (tail) arquivos de log, escutar em portas TCP/UDP, coletar do journald do systemd ou do event log do Windows. O Log Agent processa os logs, adiciona metadados (como hostname, service, source), pode aplicar regras de filtragem ou scrubbing (para remover dados sensíveis) e os envia de forma segura para o endpoint de ingestão de logs do Datadog via uma conexão TCP criptografada.

O Arquivo datadog.yaml e Suas Configurações Essenciais

Se o Datadog Agent é o corpo que coleta a telemetria, o arquivo datadog.yaml é o seu cérebro. Este arquivo de configuração central é onde você define o comportamento do Agent, habilita e desabilita funcionalidades, e especifica de onde e como os dados devem ser coletados. Uma compreensão sólida deste arquivo é fundamental para qualquer administrador de sistemas ou engenheiro de DevOps que gerencia uma implantação do Datadog.

O datadog.yaml é um arquivo de texto formatado em YAML (YAML Ain't Markup Language), um formato de serialização de dados legível por humanos. Sua localização padrão varia de acordo com o sistema operacional:

- **Linux**: /etc/datadog-agent/datadog.yaml.
- **Windows**: C:\ProgramData\Datadog\datadog.yaml.
- **macOS**: ~/.datadog-agent/datadog.yaml.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- **Docker**: dentro do contêiner, geralmente em /etc/datadog-agent/datadog.yaml. A melhor prática é montar um arquivo de configuração do host para dentro do contêiner.

O arquivo é composto por uma série de pares de chave-valor, onde as chaves de nível superior representam as principais seções de configuração do Agent. Embora o arquivo datadog.yaml contenha dezenas de opções de configuração, um subconjunto delas é considerado essencial para a operação básica e a otimização do Agent. Vamos explorar as mais críticas.

api_key: é a sua chave de autenticação única que autoriza o seu Agent a enviar dados para a sua conta específica no Datadog. Sem uma API key válida, o Forwarder não conseguirá se comunicar com o backend do Datadog.

site: o Datadog opera em múltiplos *sites* (endpoints de backend) geograficamente distribuídos para otimizar a performance e cumprir com as regulamentações de residência de dados. É crucial que esta configuração corresponda à URL do site onde sua organização foi criada. O valor padrão é datadoghq.com (US1), mas outros valores comuns incluem datadoghq.eu (EU1) , us3.datadoghq.com (US3), etc.

hostname: por padrão, o Agent tenta detectar automaticamente o hostname do host onde está rodando. No entanto, em ambientes complexos ou na nuvem, pode ser necessário definir um hostname explícito para garantir a consistência e a identificação correta do host no Datadog.

tags: são o pilar da organização e filtragem no Datadog, permite definir um conjunto de tags globais que serão aplicadas a **todas** as métricas, traces e logs coletados por aquele Agent. Esta é a maneira mais eficiente de garantir que toda a telemetria de um host seja enriquecida com metadados essenciais, como o ambiente, o serviço, a equipe proprietária, etc.

logs_enabled: coleta de logs não é habilitada por padrão em todas as instalações do Agent. Para ativar o Log Agent e começar a coletar logs, esta flag deve ser definida como true.

apm_config: controla o comportamento do Trace Agent. A configuração mais importante aqui é enabled, que ativa a recepção de traces de APM. Em instalações mais recentes, isso geralmente já vem habilitado por padrão.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

dogstatsd_port: define a porta UDP em que o Agent escutará (padrão 8125).

dogstatsd_non_local_traffic: é uma flag de segurança importante. Por padrão, ela é false, o que significa que o DogStatsD só aceitará pacotes vindos do localhost. Se você precisar enviar métricas de um contêiner para o Agent rodando no host, ou de outra máquina na rede, você precisará definir esta flag como true.

O Datadog Site: Onde Seus Dados Moram

Após a coleta, agregação e envio pelo Datadog Agent, a telemetria da sua infraestrutura e aplicações chega ao seu destino final: o Datadog Site. Este termo refere-se à plataforma de software como serviço (SaaS) do Datadog, a interface web onde seus dados são processados, armazenados, indexados e, mais importante, transformados em insights visuais e acionáveis.

O backend do Datadog é uma arquitetura massivamente escalável e multi-tenant, o que significa que os dados de múltiplos clientes (tenants) são hospedados na mesma infraestrutura, mas mantidos em silos lógicos estritamente isolados e seguros. Para otimizar a performance, a conformidade e a soberania de dados, o Datadog opera múltiplos "sites" em diferentes regiões geográficas ao redor do mundo.

Quando você cria uma conta no Datadog, sua organização é provisionada em um site específico. Os sites mais comuns são:

- **US1**: app.datadoghq.com.
- **EU1**: app.datadoghq.eu.
- **US3**: us3.datadoghq.com.
- **US5**: us5.datadoghq.com.
- **AP1**: ap1.datadoghq.com.

A escolha do site tem implicações importantes:

Soberania de Dados: se sua empresa tem requisitos legais para manter os dados dos clientes em uma região específica (como o GDPR na Europa), você deve escolher o site correspondente (ex: EU1).

Latência de Ingestão: para minimizar a latência entre sua infraestrutura e a plataforma Datadog, você deve escolher o site mais próximo de onde sua infraestrutura está hospedada.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Configuração do Agent: o site que você escolhe deve ser configurado corretamente no Datadog Agent (site: datadoghq.eu), caso contrário, o Agent não conseguirá se conectar.

Instalação do Datadog Agent em Linux

Para a execução deste laboratório, é essencial que você tenha os seguintes pré-requisitos:

- **Uma conta na AWS**: se não possuir, pode criar uma que inclui um nível de uso gratuito (Free Tier), ideal para este exercício.
- **Uma conta na plataforma Datadog**: um período de trial gratuito está disponível e pode ser ativado no site.
- **Acesso Root ou Sudo**: o script precisa de privilégios de superusuário para instalar pacotes e criar diretórios de configuração.
- **Conectividade de Rede**: o host deve ter acesso à internet para baixar o script, os pacotes e para que o Agent se comunique com o backend do Datadog.
- **API Key do Datadog**: esta é a chave de autenticação que autoriza o Agent a enviar dados para sua conta. Para encontrá-la, navegue em sua conta Datadog para Organization Settings > API Keys.
- **Site Correto**: você deve saber qual é o "site" do Datadog para sua organização (ex: datadoghq.com, datadoghq.eu).

Vamos agora instalar e validar o Agent em uma máquina virtual Linux. Recomendamos o uso de uma instância t2.micro na AWS (disponível no Free Tier) com Ubuntu 22.04.

O Datadog fornece um script de instalação automatizado que encapsula toda a complexidade de instalação, configuração e inicialização do Agent em um único comando. Este script é o método recomendado para a maioria dos usuários, pois é rápido, confiável e minimiza o risco de erros de configuração manual. Na UI do Datadog, vá em **Integrations > Agent** e selecione seu SO. O comando fornecido é a maneira mais robusta de instalar:

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

```
DD_API_KEY=<SUA_API_KEY> DD_SITE=<SEU_SITE> bash -c "$(curl -L
HTTPS://S3.AMAZONAWS.COM/DD-AGENT/SCRIPTS/INSTALL_SCRIPT.SH)"
```

O que este comando faz exatamente?

1. **DD_API_KEY=<sua-api-key>**: uma variável de ambiente que passa sua API key para o script. O script usará isso para pré-configurar o datadog.yaml.
2. **DD_SITE=<seu-site>**: uma variável de ambiente que especifica qual site do Datadog usar. Exemplos: datadoghq.com, datadoghq.eu.
3. **bash -c "..."**: executa o comando dentro das aspas usando o shell bash.
4. **curl -L** https://s3.amazonaws.com/dd-agent/scripts/install_script.sh: baixa o script de instalação da CDN do Datadog e o passa como entrada para o bash.
5. Ao final, ele inicia o serviço do Agent.

Além de DD_API_KEY e DD_SITE, o script de instalação suporta várias variáveis de ambiente opcionais para personalizar o comportamento:

Variável	Descrição	Exemplo
`DD_ENV`	Define o ambiente padrão para todas as métricas.	`DD_ENV=production`
`DD_HOSTNAME`	Define o hostname do Agent explicitamente.	`DD_HOSTNAME=meu-servidor`
`DD_TAGS`	Define tags globais, separadas por espaços.	`DD_TAGS="env:prod service:api"`
`DD_APM_ENABLED`	Habilita o Trace Agent.	`DD_APM_ENABLED=true`
`DD_LOGS_ENABLED`	Habilita a coleta de logs.	`DD_LOGS_ENABLED=true`

Figura 4 - Variáveis de ambiente
Fonte: Elaborado pelo autor (2026)

Validação e Troubleshooting Avançado

Após a instalação bem-sucedida do Datadog Agent, é crucial realizar uma série de validações para garantir que cada componente está funcionando corretamente e que a coleta de dados está ocorrendo sem problemas. Uma validação incompleta pode levar a problemas silenciosos onde o Agent parece estar rodando, mas não está

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

coletando ou enviando dados corretamente. A validação pós-instalação deve cobrir os seguintes aspectos:

1. **Processo Principal do Agent**: confirmar que o processo principal está rodando.
2. **Processos Filhos**: validar que o Trace Agent e o Process Agent foram iniciados.
3. **Conectividade de Rede**: verificar que o Agent consegue se comunicar com o backend do Datadog.
4. **Coleta de Métricas**: confirmar que o Collector está executando checks e coletando métricas.
5. **Encaminhamento de Dados**: validar que o Forwarder está enviando dados com sucesso.
6. **Portas de Rede**: confirmar que as portas esperadas estão em escuta (DogStatsD, Trace Agent, etc.).
7. **Arquivo de Configuração**: verificar que o datadog.yaml foi criado corretamente.
8. **Logs do Agent**: examinar os logs do Agent para erros ou avisos.

O Datadog fornece várias ferramentas e comandos para facilitar a validação:

- **datadog-agent status**: o comando mais importante. Fornece um relatório detalhado sobre o estado de todos os componentes do Agent.
- **datadog-agent config**: mostra a configuração atual do Agent, útil para verificar se as variáveis de ambiente foram aplicadas corretamente.
- **datadog-agent flare**: coleta logs e informações de diagnóstico do Agent em um arquivo compactado, útil para compartilhar com o suporte do Datadog.
- **systemctl status datadog-agent**: mostra o status do serviço systemd.
- **netstat ou ss**: comandos do sistema operacional para listar as portas em escuta.
- **Logs do Sistema**: arquivos de log do Agent localizados em /var/log/datadog/ (Linux) contêm mensagens de erro e avisos.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Uma validação completa deve seguir este checklist:

Item	Comando/Verificação	Resultado Esperado
Serviço Ativo	`sudo systemctl status datadog-agent`	Status `active (running)`
Processo Principal	`ps aux | grep agent`	Processo `/opt/datadog-agent/bin/agent/agent run` presente
Trace Agent	`ps aux | grep trace-agent`	Processo `trace-agent` presente
Portas em Escuta	`sudo netstat -tulnp | grep agent`	Portas 5001, 8125 (UDP), 8126 (TCP) em escuta
Configuração Carregada	`sudo datadog-agent config`	API key e site exibidos corretamente
Status Detalhado	`sudo datadog-agent status`	Collector, Forwarder, DogStatsD com status OK
Conectividade	Verificar no Datadog UI	Host aparece em Infrastructure > Host List
Métricas Coletadas	Verificar no Datadog UI	Métricas de CPU, memória, disco visíveis
Logs do Agent	`sudo tail -f /var/log/datadog/agent.log`	Sem erros críticos

Figura 5 – Checklist para validação
Fonte: Elaborado pelo autor (2026)

Durante a validação, você pode encontrar alguns problemas comuns:

API Key Inválida: o Forwarder reportará erros de autenticação. Verifique se a API key foi copiada corretamente.

Site Incorreto: o Agent tentará enviar dados para o endpoint errado. Verifique a configuração site: no datadog.yaml.

Conectividade de Rede: se o Agent não conseguir alcançar o backend do Datadog, pode haver um firewall bloqueando a porta HTTPS 443. Verifique as regras de firewall.

Permissões de Arquivo: alguns checks podem falhar se o Agent não tiver permissões para ler certos arquivos do sistema. Verifique os logs para mensagens de permissão negada.

Configuração do Agent em Containers Docker

A containerização com Docker revolucionou a forma como desenvolvemos, testamos e implantamos aplicações. No entanto, ela também introduz novos desafios para a observabilidade. Diferente de um host tradicional onde você instala o Agent uma vez e ele monitora tudo, em um ambiente Docker você pode ter centenas ou

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

milhares de contêineres efêmeros, cada um potencialmente executando uma aplicação diferente, sendo criados e destruídos dinamicamente.

O Datadog Agent foi projetado para funcionar perfeitamente em ambientes containerizados. Ele pode ser executado como um contêiner separado (geralmente em um padrão chamado "sidecar" ou "daemon set" em orquestradores como Kubernetes) e coletar dados de todos os outros contêineres no host. Além disso, o Agent possui um recurso poderoso chamado **Autodiscovery**, que permite que ele detecte automaticamente novos contêineres, identifique quais serviços estão rodando neles e aplique as configurações de monitoramento apropriadas, tudo sem intervenção manual.

Executando o Datadog Agent como um Contêiner Docker

Executar o Datadog Agent como um contêiner Docker oferece várias vantagens em relação à instalação em um host tradicional. O Agent é fornecido como uma imagem Docker pré-construída, disponível no Docker Hub e no Google Container Registry (GCR), permitindo uma implantação rápida e consistente. Por Que Executar o Agent em Docker?

1. **Consistência**: a mesma imagem do Agent funciona em qualquer ambiente que tenha Docker instalado, eliminando problemas de "funciona na minha máquina".
2. **Isolamento**: o Agent roda em seu próprio contêiner, isolado de outras aplicações, reduzindo o risco de conflitos.
3. **Escalabilidade**: em orquestradores como Kubernetes, você pode facilmente escalar o número de instâncias do Agent.
4. **Simplicidade**: não há necessidade de gerenciar repositórios de pacotes ou dependências do sistema.

O Datadog fornece imagens Docker oficiais para o Agent em múltiplos registros, A imagem é baseada em uma distribuição Linux leve (geralmente Alpine ou Debian slim) e inclui todos os componentes do Agent (Collector, Forwarder, DogStatsD, Trace Agent, Log Agent).

- **Docker Hub:** datadog/agent:latest (sujeito a limites de taxa).
- **Google Container Registry:** gcr.io/datadoghq/agent:latest (recomendado).

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- **Amazon ECR:** public.ecr.aws/datadog/agent:latest.

O comando mais simples para executar o Agent em Docker é:

```
docker run -d \
  --name datadog-agent \
  -e DD_API_KEY=<sua-api-key> \
  -e DD_SITE=<seu-site> \
  gcr.io/datadoghq/agent:latest
```

- -d: executa o contêiner em modo detached (background).
- --name datadog-agent: nomeia o contêiner para fácil referência.
- -e DD_API_KEY=...: define a variável de ambiente com sua API key.
- -e DD_SITE=...: define o site do Datadog.
- gcr.io/datadoghq/agent:latest: a imagem a usar.

Quando executando o Agent em Docker, a configuração é feita principalmente através de variáveis de ambiente (em vez de editar datadog.yaml). As variáveis essenciais são:

Variável	Descrição	Exemplo
DD_API_KEY	Sua chave de API do Datadog (obrigatória).	DD_API_KEY=a1b2c3d4...
DD_SITE	O site do Datadog.	DD_SITE=datadoghq.com
DD_ENV	Ambiente padrão para métricas.	DD_ENV=production
DD_HOSTNAME	Hostname do Agent.	DD_HOSTNAME=my-app-01
DD_TAGS	Tags globais, separadas por espaços.	DD_TAGS="env:prod service:api"
DD_APM_ENABLED	Habilita o Trace Agent.	DD_APM_ENABLED=true
DD_LOGS_ENABLED	Habilita a coleta de logs.	DD_LOGS_ENABLED=true
DD_DOGSTATSD_NON_LOCAL_TRAFFIC	Permite DogStatsD de fora do localhost.	DD_DOGSTATSD_NON_LOCAL_TRAFFIC=true

Figura 6 – Variáveis de ambiente
Fonte: Elaborado pelo autor (2026)

Um aspecto crítico ao executar o Agent em Docker é entender como a rede funciona:

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Contêineres na Mesma Rede: se suas aplicações estão em contêineres na mesma rede Docker que o Agent, elas podem se comunicar com o Agent usando o hostname do contêiner (ex: datadog-agent:8125 para DogStatsD).

Host Network: se você usar --network host, o contêiner compartilha a pilha de rede do host, permitindo que aplicações no host se comuniquem com o Agent via localhost:8125.

Contêineres em Redes Diferentes: se os contêineres estão em redes diferentes, você pode precisar usar --network bridge e mapear portas explicitamente.

Para que o Agent possa coletar dados de outros contêineres e do host, ele precisa acessar certos diretórios e sockets do host. Isso é feito através de montagens de volume:

```
docker run -d \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -v /proc:/host/proc:ro \
  -v /sys:/host/sys:ro \
  ...
```

Esses volumes permitem que o Agent:

- Acesse o socket do Docker (/var/run/docker.sock) para detectar contêineres.
- Leia informações de processo do host (/proc e /sys).

Por padrão, os dados do Agent (como cache de métricas) são armazenados em memória. Para persistência, você pode montar um volume para o diretório de dados do Agent:

```
docker run -d \
  -v /var/lib/datadog-agent:/var/lib/datadog-agent \
  ...
```

Montagem dos Volumes Essenciais

Para que o Datadog Agent, executando dentro de um contêiner Docker, possa coletar dados de outros contêineres e do host, ele precisa acessar certos arquivos e

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

sockets do host. Isso é feito através da montagem de volumes (ou bind mounts). Entender quais volumes montar e por quê é crucial para uma implantação bem-sucedida do Agent em Docker.

Um volume em Docker é um mecanismo para persistir dados fora do contêiner ou para compartilhar dados entre o host e o contêiner. Para o Datadog Agent, usamos principalmente bind mounts para acessar dados do host. Existem três tipos principais:

- **Named Volumes**: gerenciados pelo Docker em /var/lib/docker/volumes/.
- **Bind Mounts**: montam um arquivo ou diretório específico do host dentro do contêiner.
- **tmpfs Mounts**: armazenam dados em memória.

Volumes Essenciais para o Datadog Agent:

- **/var/run/docker.sock:** este é o socket do daemon do Docker. O Agent usa este socket para se comunicar com o Docker e descobrir quais contêineres estão rodando no host.
- **/proc e /sys:** estes diretórios contêm informações do sistema operacional sobre processos, memória, CPU e outros recursos. O sufixo :ro significa "read-only" (somente leitura), uma boa prática de segurança.
- **`/etc/passwd:`** este arquivo contém informações sobre os usuários do sistema. O Agent o lê para enriquecer os dados de processo com nomes de usuários.
- **/var/lib/datadog-agent:** este diretório armazena dados persistentes do Agent, como cache de métricas e estado do Collector.

Um comando completo com todos os volumes essenciais se parece com:

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

```
docker run -d \
  --name datadog-agent \
  -e DD_API_KEY=<sua-api-key> \
  -e DD_SITE=<seu-site> \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -v /proc:/host/proc:ro \
  -v /sys:/host/sys:ro \
  -v /etc/passwd:/etc/passwd:ro \
  -v /var/lib/datadog-agent:/var/lib/datadog-agent \
  gcr.io/datadoghq/agent:latest
```

Além de volumes, você pode precisar mapear portas do contêiner para o host, permitindo que outras aplicações (em contêineres ou no host) se comuniquem com o Agent:

```
docker run -d \
  -p 8125:8125/udp \
  -p 8126:8126/tcp \
  ...
```

- -p 8125:8125/udp: mapeia a porta UDP 8125 do contêiner para a porta 8125 do host (DogStatsD).
- -p 8126:8126/tcp: mapeia a porta TCP 8126 do contêiner para a porta 8126 do host (Trace Agent).

Autodiscovery: Detecção e Monitoramento Automático de Contêineres

Um dos recursos mais poderosos do Datadog Agent em ambientes containerizados é o Autodiscovery. Este recurso permite que o Agent detecte automaticamente novos contêineres conforme eles são criados, identifique quais serviços estão rodando (como Redis, PostgreSQL, NGINX, etc.) e aplique automaticamente as configurações de monitoramento apropriadas, tudo sem intervenção manual.

Em um ambiente tradicional, você instalaria o Agent uma vez em um host e configuraria manualmente quais serviços monitorar através de arquivos de configuração. Isso funciona bem quando você tem um número fixo e conhecido de serviços.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Porém, em um ambiente containerizado moderno:

- Contêineres são criados e destruídos dinamicamente.
- Você pode ter centenas ou milhares de contêineres em um cluster Kubernetes.
- Novos serviços são implantados continuamente.
- A configuração manual para cada contêiner é impraticável.

O Autodiscovery resolve este problema permitindo que o Agent descubra automaticamente quais contêineres estão rodando e configure o monitoramento dinamicamente. O Autodiscovery opera em três etapas:

- **Detecção de Contêineres:** o Agent monitora o socket do Docker (ou a API do Kubernetes) para detectar quando novos contêineres são criados ou quando contêineres existentes são destruídos. Quando um novo contêiner é detectado, o Agent coleta metadados sobre ele.
- **Identificação de Serviço:** o Agent tenta identificar qual serviço está rodando no contêiner. Isso é feito através de:
  - **Labels do Contêiner:** o contêiner pode ter labels que indicam qual serviço está rodando (ex: com.datadog.ad.check.id=redis).
  - **Variáveis de Ambiente:** o contêiner pode ter variáveis de ambiente que indicam o tipo de serviço.
  - **Porta Padrão:** o Agent pode tentar conectar-se a portas padrão conhecidas (ex: 6379 para Redis, 5432 para PostgreSQL).
- **Aplicação de Configuração:** uma vez que o serviço é identificado, o Agent aplica automaticamente a configuração de check apropriada. Por exemplo, se um contêiner Redis é detectado.

A forma mais comum de usar o Autodiscovery é através de labels de contêiner. Você adiciona labels especiais ao seu contêiner Docker que instruem o Agent sobre como monitorá-lo. Exemplo: Para monitorar um contêiner Redis:

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

```
docker run -d \
  --name my-redis \
  -l com.datadog.ad.check.id=redis \
  -l "com.datadog.ad.check.redis.instances=[{\"host\":
\"localhost\", \"port\": 6379}]" \
  redis:latest
```

Alternativamente, você pode fornecer configurações de Autodiscovery através de um arquivo YAML montado no Agent, o Agent substitui `%%host%%` e `%%port%%` pelos valores detectados do contêiner:

```
# /etc/datadog-agent/conf.d/redis.d/auto_conf.yaml
ad_identifiers:
  - redis

init_config:

instances:
  - host: "%%host%%"
    port: "%%port%%"
```

Validação Completa Pós-Instalação em Docker

Assim como na instalação em Linux, após executar o Datadog Agent em Docker, é crucial realizar uma série de validações para garantir que o Agent está funcionando corretamente e coletando dados como esperado. A validação em Docker tem algumas particularidades, pois você está trabalhando com um contêiner isolado.

Uma validação completa em Docker deve cobrir os seguintes aspectos:

Item	Método de Verificação	Resultado Esperado
Contêiner Ativo	`docker ps | grep datadog-agent`	Contêiner listado com status `Up`
Logs Sem Erros	`docker logs datadog-agent`	Mensagens de inicialização, sem erros críticos
Volumes Montados	`docker inspect datadog-agent | grep Mounts`	Todos os volumes esperados listados
Portas Mapeadas	`docker port datadog-agent`	Portas 8125 (UDP) e 8126 (TCP) mapeadas
Status Interno	`docker exec datadog-agent agent status`	Collector, Forwarder, DogStatsD com status OK
Conectividade	`docker exec datadog-agent agent config`	API key e site exibidos corretamente
Detecção de Contêineres	`docker exec datadog-agent docker ps`	Contêineres no host listados
Dados no Datadog	Verificar no Datadog UI	Host aparece em Infrastructure > Host List

Figura 7 – Validação completa em Docker

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Fonte: Elaborado pelo autor (2026)

Além das validações gerais, há algumas verificações específicas para ambientes Docker:

- Verificar o Acesso ao Socket do Docker.
  - docker exec datadog-agent docker ps.
- Verificar o Mapeamento de Portas.
  - Testar DogStatsD..
  - echo "test.metric:1|c" | nc -u -w0 127.0.0.1 8125
  - Testar Trace Agent.

    curl **http://127.0.0.1:8126/info**
- Verificar a Persistência de Dados.
  - docker exec datadog-agent ls -la /var/lib/datadog-agent/
- Verificar Limites de Recursos.
  - docker stats datadog-agent.

Explorando Dados de Contêineres no Datadog

Quando você executa o Datadog Agent em Docker, ele coleta não apenas métricas de infraestrutura do host, mas também metadados e métricas específicas de contêineres. A interface do Datadog fornece visualizações especializadas para explorar dados de contêineres de forma eficiente.

A seção Infrastructure > Containers é dedicada à visualização de dados de contêineres.

- **Container List:** uma lista tabular de todos os contêineres monitorados, mostrando.
- **Container Map:** similar ao Host Map, mas para contêineres. Cada contêiner é representado como um quadrado, e a cor indica seu status.

O Datadog coleta várias métricas específicas de contêineres:

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Métrica	Descrição
`docker.cpu.percentage`	Percentual de CPU usado pelo contêiner.
`docker.memory.usage`	Memória usada pelo contêiner.
`docker.mem.limit`	Limite de memória do contêiner.
`docker.net.bytes_sent`	Bytes enviados pela rede.
`docker.net.bytes_rcvd`	Bytes recebidos pela rede.
`docker.io.read_bytes`	Bytes lidos do disco.
`docker.io.write_bytes`	Bytes escritos no disco.

Figura 8 – Dados de Contêineres
Fonte: Elaborado pelo autor (2026)

Quando o Agent coleta dados de um contêiner, ele automaticamente adiciona tags que permitem filtrar e agrupar por:

- container_id: o ID do contêiner.
- container_name: o nome do contêiner.
- image_name: o nome da imagem.
- image_tag: a tag da imagem.
- Qualquer label customizado que você adicionar ao contêiner.

Um ponto forte do Datadog é a capacidade de correlacionar dados de contêineres com dados do host. Esta correlação é particularmente útil para troubleshooting de problemas de performance. Por exemplo:

- Se um contêiner está consumindo muita CPU, você pode ver qual é o impacto na CPU do host.
- Se um contêiner está tendo problemas de rede, você pode ver se há problemas de rede no host também.

Se você está usando Kubernetes, o Datadog fornece visualizações ainda mais avançadas, incluindo:

- **Pods**: visualizar e monitorar pods Kubernetes.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- **Deployments**: monitorar o status de deployments.
- **Services**: monitorar serviços Kubernetes.
- **Nodes**: monitorar nós do cluster.

Métricas de Infraestrutura

O Datadog Agent coleta centenas de métricas de infraestrutura por padrão. No entanto, um subconjunto dessas métricas é fundamental para entender a saúde e a performance de qualquer sistema. Vamos explorar as quatro categorias mais importantes.

Agora que o Agent está coletando dados, vamos para a UI do Datadog e entender o que estamos vendo. Vá para Infrastructure > Infrastructure List e clique no host que você acabou de adicionar.

Você será levado ao dashboard de host padrão, que contém os indicadores de saúde mais críticos para um sistema Linux. Vamos analisar os quatro principais.

1 - CPU

O que é: a porcentagem de tempo que a CPU passou trabalhando.

Métricas no Datadog:

system.cpu.user: tempo gasto executando código de aplicação (no espaço do usuário). Um valor alto aqui geralmente significa que sua aplicação está ocupada.

system.cpu.system: tempo gasto executando código do kernel em nome da sua aplicação (ex: fazendo chamadas de sistema, I/O de rede). Um valor alto aqui pode indicar uma aplicação que faz muitas chamadas de sistema ou I/O.

system.cpu.iowait: tempo que a CPU passou ociosa, esperando por uma operação de I/O (geralmente disco) ser concluída. Um valor alto e persistente aqui é um forte indicador de um gargalo de disco. A CPU quer trabalhar, mas não pode porque está esperando dados do disco.

system.cpu.idle: em ambientes virtualizados (nuvem), este é o tempo que a CPU estava disponível para a sua VM, mas foi "roubada" pelo hypervisor para ser usada por outra VM. Um valor alto aqui significa que você está em um "vizinho barulhento" e sua instância não está recebendo os ciclos de CPU que deveria.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

2 - Memória

O que é: a quantidade de memória RAM que está sendo usada.

Métricas no Datadog: é um erro comum olhar para system.mem.free e entrar em pânico quando ele está baixo. O Linux é agressivo ao usar a memória livre para cache de disco (cached e buffered) para acelerar as operações. A métrica mais importante é system.mem.usable, que representa a memória que está verdadeiramente disponível para as aplicações (a soma de free + cached + buffered). Um problema real de memória ocorre quando system.mem.usable está baixo e o sistema começa a fazer **swap** (mover páginas de memória para o disco), o que é extremamente lento. Monitore system.swap.used - se este valor começar a subir, você tem um problema de memória.

3 - Saturação: Load Average

O que é: uma medida da "demanda" por CPU. É o número médio de processos na fila de execução (rodando ou esperando para rodar) durante um período de tempo.

Métricas no Datadog: a regra geral é que o load average não deve exceder o número de cores de CPU do seu sistema. Se você tem uma máquina de 4 cores e um system.load.1 de 8.0, significa que, em média, 4 processos estavam esperando na fila enquanto 4 estavam rodando. Isso indica que o sistema está saturado; há mais trabalho a ser feito do que a CPU consegue lidar. Comparar as médias de 1, 5 e 15 minutos ajuda a entender a tendência: se a média de 1 minuto é muito maior que a de 15, a carga está aumentando rapidamente.

4 - I/O de Disco

O que é: a atividade de leitura e escrita nos seus dispositivos de armazenamento.

Métricas no Datadog: picos de r_s ou w_s são normais. O problema é quando o await começa a subir. Um await alto, especialmente se correlacionado com um system.cpu.iowait alto, é um sinal claro de que seu disco não está conseguindo acompanhar a demanda. Isso pode ser causado por um hardware lento (HDD em vez de SSD) ou por um volume de operações simplesmente muito alto.

- system.io.r_s (leituras por segundo);
- system.io.w_s (escritas por segundo);

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- system.io.avg_rq_sz (tamanho médio das requisições);
- system.io.await (tempo médio de espera por uma operação de I/O).

Nenhuma dessas métricas existe isoladamente. A chave para a análise de performance é correlacioná-las:

- Alta Utilização de CPU + Alto Load Average: o sistema está sobrecarregado de CPU.
- Baixa Utilização de CPU + Alto Load Average + Alto system.cpu.iowait: o sistema está com um gargalo de I/O.
- Alta Utilização de Memória + Uso de Swap: o sistema está sem memória RAM.
- Alta Utilização de Disco + Alto system.cpu.iowait: o disco está saturado.

O Método USE: Uma Abordagem Sistemática para Análise de Performance

O Método USE (Utilization, Saturation, Errors) é uma metodologia desenvolvida por Brendan Gregg, um dos maiores especialistas em performance de sistemas do mundo. Ele fornece um framework simples e poderoso para analisar a performance de qualquer recurso de um sistema (CPU, memória, disco, rede, etc.).

O objetivo do método USE é garantir que você não se esqueça de verificar nenhuma área importante ao investigar um problema de performance. Ele é projetado para ser rápido e eficiente, permitindo que você identifique rapidamente a causa raiz de um problema.

Para cada recurso do sistema, o método USE nos instrui a verificar três tipos de métricas:

1 - Utilization (Utilização): o percentual de tempo que o recurso esteve ocupado. Uma utilização de 100% significa que o recurso esteve ocupado o tempo todo.

2 - Saturation (Saturação): a medida em que o recurso tem trabalho extra que não consegue atender, geralmente resultando em uma fila. Uma saturação alta indica que o recurso é um gargalo.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

3 - Errors (Erros): o número de erros que ocorreram no recurso. Erros podem indicar problemas de hardware, software ou configuração.

Vamos ver como o método USE se aplica aos principais recursos de um sistema:

CPU

Utilization: system.cpu.user, system.cpu.system, system.cpu.idle.

Saturation: system.load.1 (Load Average). Um load maior que o número de núcleos indica saturação.

Errors: erros de CPU são raros e geralmente indicam problemas de hardware.

Memória

Utilization: system.mem.used, system.mem.pct_usable.

Saturation: system.swap.used. o uso de swap é um sinal claro de saturação de memória.

Errors: erros de memória (como OOM - Out Of Memory) são geralmente encontrados nos logs do sistema.

Disco

Utilization: system.disk.in_use.

Saturation: comprimento da fila de I/O (disponível em métricas mais avançadas).

Errors: erros de disco (como erros de leitura/escrita) são geralmente encontrados nos logs do sistema.

Rede

Utilization: system.net.bytes_sent, system.net.bytes_rcvd.

Saturation: comprimento da fila de pacotes (disponível em métricas mais avançadas).

Errors: system.net.packets_out.error, system.net.packets_in.error.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

MERCADO, CASES E TENDÊNCIAS

Reconhecimento do mercado

O Datadog foi novamente reconhecido pela Gartner® como *Líder* em **Digital Experience Monitoring**. A plataforma destaca a importância de IA, escalabilidade e controle de custos ao oferecer uma solução de observabilidade unificada. Link

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Você aprendeu o conceito fundamental que separa equipes que apenas monitoram das que realmente entendem seus sistemas: **observabilidade como ferramenta exploratória**, capaz de revelar falhas inesperadas em ambientes complexos. Conheceu profundamente os três pilares — métricas (com foco em alta cardinalidade), logs (estruturados vs. não estruturados) e traces — e viu como eles se conectam dentro da plataforma.

Exploramos a **arquitetura lógica do Datadog**, seus pipelines de ingestão, armazenamento, correlação e visualização. No nível técnico, você entendeu a **arquitetura interna do Datadog Agent**, seus subprocessos e o papel de cada um na coleta de telemetria.

No lado prático, você instalou e validou o Agent em Linux e Docker, aprendeu a diagnosticar problemas, entendeu como o Agent coleta métricas por host, container e processo — e viu como interpretar sinais vitais como CPU, memória, load average e I/O para identificar gargalos reais.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

REFERÊNCIAS

DATADOG. **Datadog Agent Overview**. Datadog Documentation. Disponível em: https://docs.datadoghq.com/ . Acesso em: 08 jan. 2026.

DATADOG. **Docker Autodiscovery.** Datadog Documentation. Disponível em: https://docs.datadoghq.com/getting_started/containers/autodiscovery/?tab=adannotationsv2agent736. Acesso em: 08 jan. 2025.

GREGG, B. **The USE Method**. 2013. Disponível em: https://www.brendangregg.com/usemethod.html. Acesso em: 08 jan. 2025.

KÁLMAN, R. E. **On the general theory of control systems**. Proceedings of the First International Congress on Automatic Control. Moscow. 1960

SRIDHARAN, C. **Distributed Systems Observability: A Guide to Building Robust Systems**. O'Reilly Media. 2018.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

PALAVRAS-CHAVE

Agent, Host, Container

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com



PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com